ილიას სახელმწიფო უნივერსიტეტი

კომპიუტერული გრაფიკის ალგორითმები

გიორგი ფარცხალაძე

ლექცია 5:

სამგანზომილებიანი გრაფიკა

სამგანზომილებიანი გრაფიკა (3D)

სხეულების მოდელების 3 ძირითადი ტიპი:

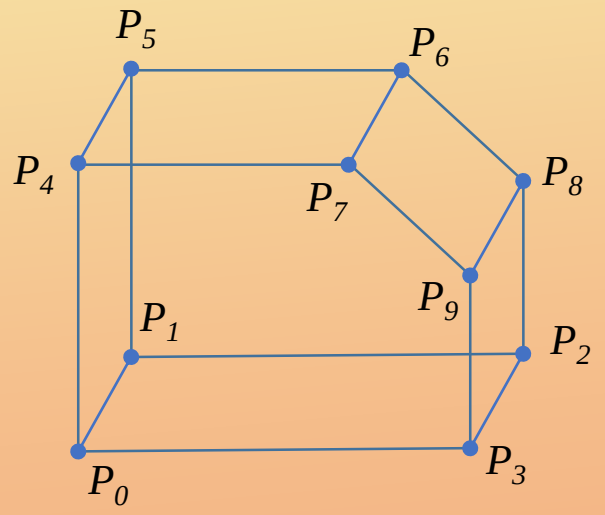
1. „მავთულიანი მოდელი“ (Wireframe model)

2. „ზედაპირების მოდელი (Surface model)

3. „მოცულობითი მოდელი“ (Volume (volumetric, solid) model)

1. მავთულიანი მოდელი.

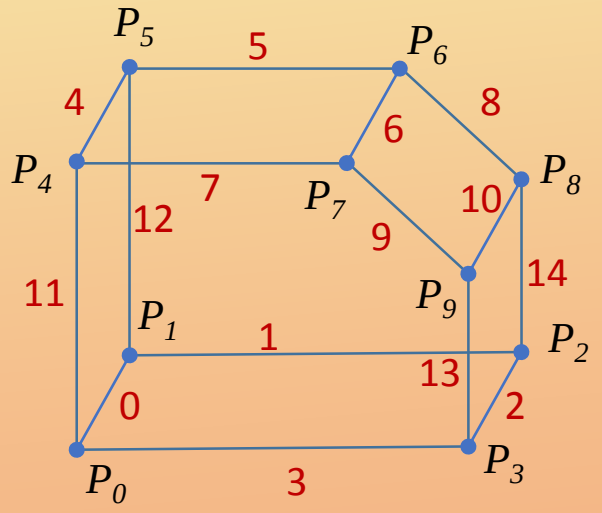
გამოიყენება სწრაფი ვიზუალიზაციისთვის, სადაც არ აქვს მნიშვნელობა ზედაპირების ტექსტურებს და არ არის აუცილებელი უხილავი (სხვა სხეულებით დაფარულის, ან ობიექტის უკანა მხარეს მდებარე) ზედაპირების დამალვა.



მავთულიანი (Wireframe) მოდელის სტრუქტურა:

(ჰგავს ვექტორულ გრაფიკას): შედგება ე.წ. „პრიმიტივებისგან“, ანუ მარტივი გეომეტრიული ფიგურებისგან, როგორიც არის: ხაზი (მონაკვეთი), რკალი, წრეწირი, ოვალი, პოლილაინი, სპლაინი.

ხშირად კონსტრუქცია შეიცავს ცალ-ცალკე შენახულ გეომეტრიასა და ტოპოლოგიას გეომეტრია განსაზღვრავს ზომებს (კოორდინატებს), ხოლო ტოპოლოგია - კავშირებს



წვეროები			
	x	y	z
P0	0	0	0
P1	0	0	40
P2	110	0	40
P3	110	0	0
P4	0	75	0
P5	0	75	40
P6	60	75	40
P7	60	75	0
P8	110	40	40
P9	110	40	0

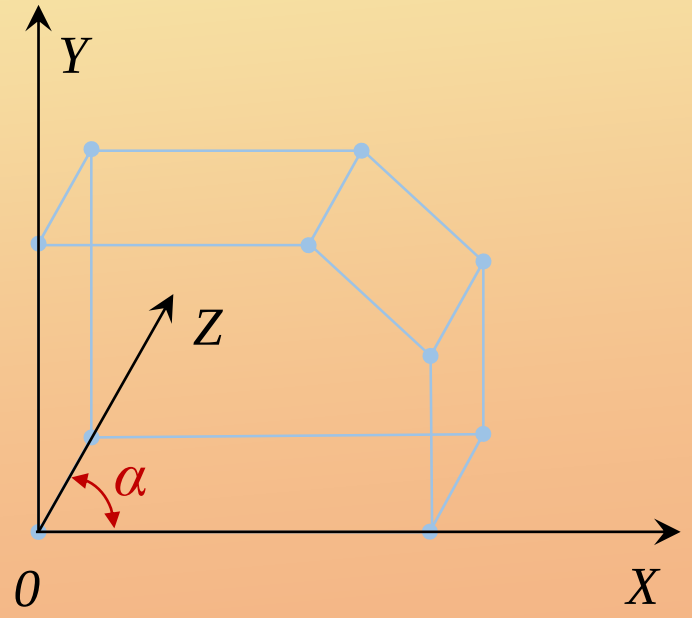
⇔

წიბოები		
L0	*P0*	*P1*
L1	*P1*	*P2*
L2	*P2*	*P3*
L3	*P3*	*P0*
L4	*P4*	*P5*
L5	*P5*	*P6*
L6	*P6*	*P7*
L7	*P7*	*P4*
L8	*P6*	*P8*
L9	*P7*	*P9*
L10	*P8*	*P9*
L11	*P0*	*P4*
L12	*P1*	*P5*
L13	*P3*	*P9*
L14	*P2*	*P8*

სამგანზომილებიან გრაფიკაში, 2D გრაფიკისგან განსხვავებით, დიდი მნიშვნელობა აქვს პროექციას, ანუ იმას, როგორ ხდება გამოსახულების გამოტანა ეკრანზე - რა რაკურსით, რა კუთხებით, რა პერსპექტივით და ა.შ.

აქვე შევეცდებით მარტივი 3D - პროექციის შექმნა. გამოვიყენოთ ე.წ. ფრონტალურ-დიმეტრიული პროექცია, რომელშიც XOY ზედაპირი აისახება პირდაპირ, ხოლო Z ღერძი - გარკვეული კუთხით α

ზომები სამივე ღერძის გასწვრივ დაიტანება შეუცვლელად

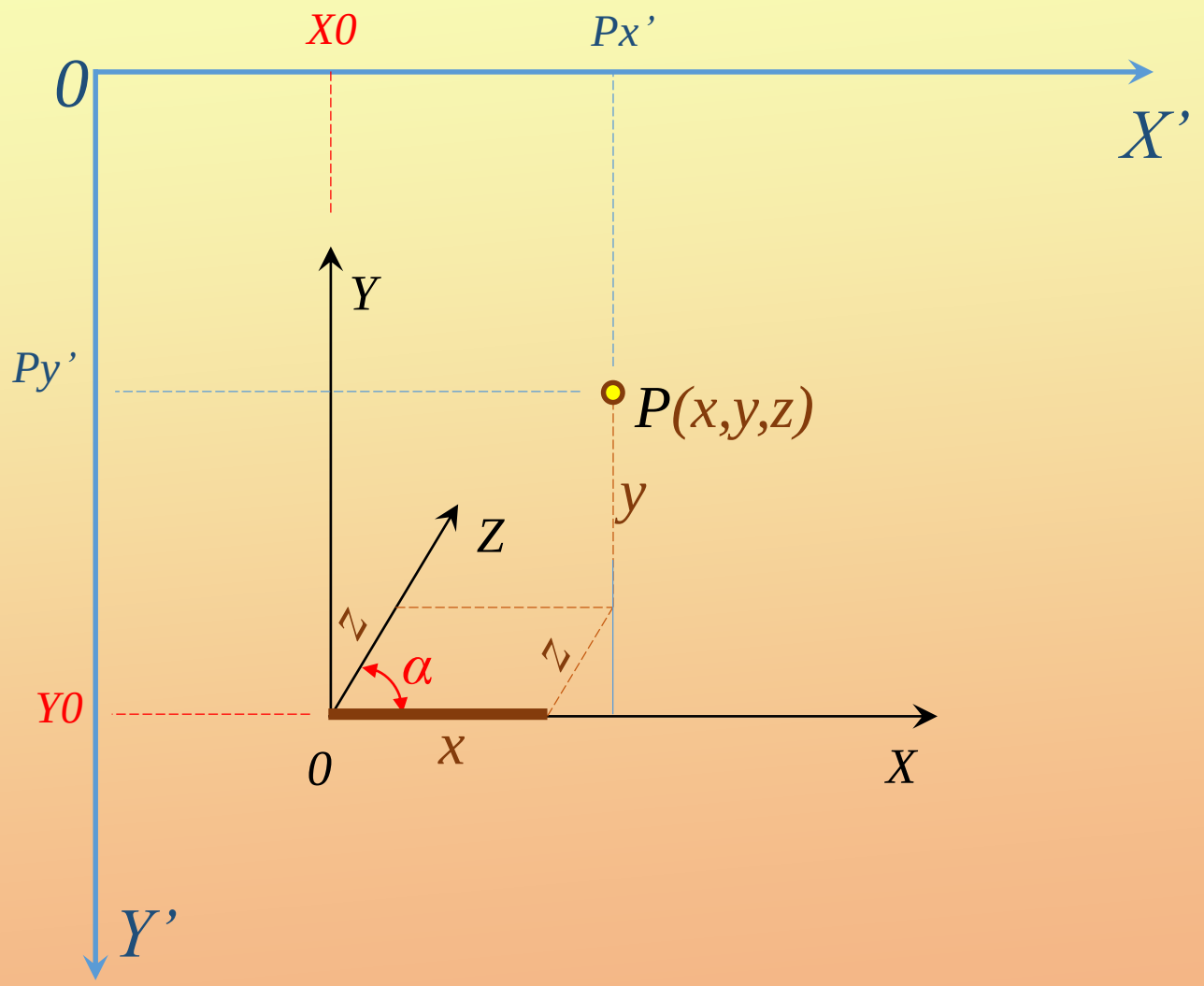


საჭიროა სამგანზომილებიანი სივრცის (XYZ) გადაყვანა ეკრანის კოორდინატებში (X'OY')

სივრცობრივი წერტილი P(x,y,z)-ს ეკრანის კოორდინატები იქნება:

$$Px' = X0 + x + z*\cos(\alpha)$$

$$Py' = Y0 - y - z*\sin(\alpha)$$



გრაფიკული ობიექტების ეკრანზე გამოტანისთვის დაგვჭირდება სპეციალური ფუნქციები. ყველა შემთხვევაში ამ ფუნქციებს გადაეცემათ დიმეტრიული პროექციის პარამეტრები. ეს იქნება კოორდინატთა საწყისი წერტილის X0,Y0 კოორდინატები და კუთხე α

იმისთვის რომ არ მოგვიწიოს ყოველთვის ამ პროექციის პარამეტრების ცალ-ცალკე გადაცემა ფუნქციებში, შევქმნათ ახალი მონაცემთა სტრუქტურა:

```
struct pr_3d_dimetric // ახალი მონაცემთა ტიპი. დიმეტრიული პროექციის პარამეტრები
  {
        int X0;
        int Y0;
        double alpha;
  };
```

მონაკვეთის (x0,y0,z0)-(x1,y1,z1) გამოხაზვა ფერში(rr,gg,bb) ხორციელდება ფუნქციით:

```
void draw_line_3d(pr_3d_dimetric my_win, int x0, int y0, int z0, int x1, int y1, int z1, int rr, int gg, int bb)
  {
        int d_x0; int d_y0;
        int d_x1; int d_y1;

        // z0 = z0/2; z1 = z1/2;  //tu gvinda rom realisturobistvis Z – gherdze zomebi sheikumshos…

        d_x0 = my_win.X0 + x0 + z0*cos(my_win.alpha);
        d_y0 = my_win.Y0 - y0 - z0*sin(my_win.alpha);
        d_x1 = my_win.X0 + x1 + z1*cos(my_win.alpha);
        d_y1 = my_win.Y0 - y1 - z1*sin(my_win.alpha);

    setcolor(COLOR(rr, gg, bb));
    line (d_x0, d_y0, d_x1, d_y1);
  }
```

შემდეგ ვაყალიბებთ სტრუქტურებს წვეროებისთვის და წიბოებისთვის:

```
struct vertex
  {
        int num;
        double x;
        double y;
        double z;
  };

class linie
 {
       public:
       int num;
       vertex a;
       vertex b;
       double len(); // ეს ფუნქცია - მეთოდი ანგარიშობს მონაკვეთის სიგრძეს
 };
 double linie::len()
 {
       return sqrt( (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y) + (a.z-b.z)*(a.z-b.z));
 }
```

```
int main()
 {
   initwindow(800,800);
   setbkcolor(COLOR(255,255,255));
   cleardevice();

           // parametrebi dimetriuli proeqciisTvis:
           double kutxe = 0.7;   // Z - gherdzis gamosaxulebis kutxe
           int XX0 = 250; int YY0 = 600;    // koordinatebis sawyisi wertilis adgili

           pr_3d_dimetric win1;  // win1 - cvladi, romelshic inaxeba proeqciis parametrebi
           win1.alpha = kutxe; win1.X0 = XX0; win1.Y0 = YY0;

           draw_line_3d ( win1, 0, 0, 0, 300, 0, 0, 255, 0, 0 ); // sakoordinato xazebis gamoxazva:
           draw_line_3d ( win1, 0, 0, 0, 0, 300, 0, 0, 255, 0 );
           draw_line_3d ( win1, 0, 0, 0, 0, 0, 300, 0, 0, 255 );

           vertex points[10];    // 10 wertili
           linie xazi[15];       // 15 xazi

//// geometria:
           int i;
           for (i=0; i<10; i++) points[i].num = i;               // wertilebis danomrva

           points[0].x = 0; points[0].y = 0; points[0].z = 0;
           points[1].x = 0; points[1].y = 0; points[1].z =  40;
           points[2].x = 110; points[2].y = 0; points[2].z = 40;
           points[3].x = 110; points[3].y = 0; points[3].z = 0;

           points[4].x = 0; points[4].y =  75; points[4].z = 0;
           points[5].x = 0; points[5].y = 75; points[5].z =  40;
           points[6].x =  60; points[6].y = 75; points[6].z = 40;
           points[7].x =  60; points[7].y = 75; points[7].z = 0;

           points[8].x =  110; points[8].y = 40; points[8].z = 40;
           points[9].x =  110; points[9].y = 40; points[9].z = 0;
```

```
//// topologia:
          for (i=0; i<15; i++) xazi[i].num = i;      // xazebis danomrva

          xazi[0].a = points[0];  xazi[0].b = points[1];
          xazi[1].a = points[1];  xazi[1].b = points[2];
          xazi[2].a = points[2];  xazi[2].b = points[3];
          xazi[3].a = points[3];  xazi[3].b = points[0];

          xazi[4].a = points[4];  xazi[4].b = points[5];
          xazi[5].a = points[5];  xazi[5].b = points[6];
          xazi[6].a = points[6];  xazi[6].b = points[7];
          xazi[7].a = points[7];  xazi[7].b = points[4];

          xazi[8].a = points[6];  xazi[8].b = points[8];
          xazi[9].a = points[7];  xazi[9].b = points[9];
          xazi[10].a = points[8];  xazi[10].b = points[9];

          xazi[11].a = points[0];  xazi[11].b = points[4];
          xazi[12].a = points[1];  xazi[12].b = points[5];
          xazi[13].a = points[3];  xazi[13].b = points[9];
          xazi[14].a = points[2];  xazi[14].b = points[8];

          for (i=0; i<15; i++)
            {
               printf ("\n %d-s sigrdze %g-a; ",xazi[i].num, xazi[i].len() );
               draw_line_3d ( win1, xazi[i].a.x, xazi[i].a.y, xazi[i].a.z, xazi[i].b.x, xazi[i].b.y,  xazi[i].b.z, 55, 10, 0 );
            }
  getch();
  closegraph();
}
```

„მავთულიანი“ (wireframe) მოდელების

უპირატესობები:

ა. ვიზუალიზაციის სისწრაფე (ნებისმიერი სირთულის სცენის მოძრაობა რეალურ დროში)

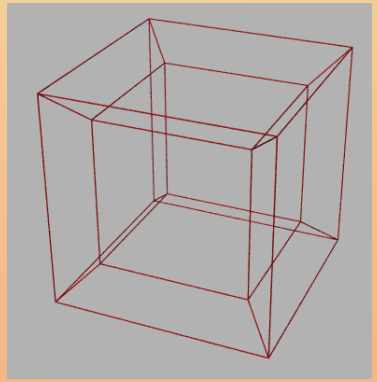
ბ. გამჭვირვალობა (კარგად ჩანს ყველა ელემენტი)

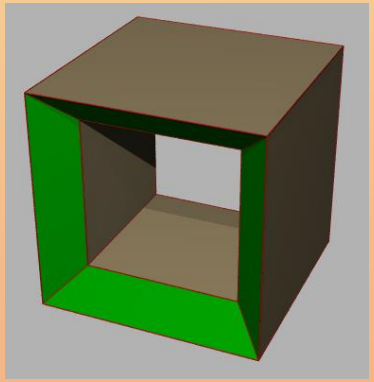
ნაკლები:

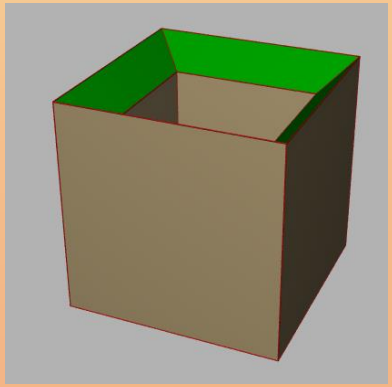
ა. ტექსტურების დადების შეუძლებლობა

ბ. რთული სცენის შემთხვევაში გაძნელებული ორიენტირება

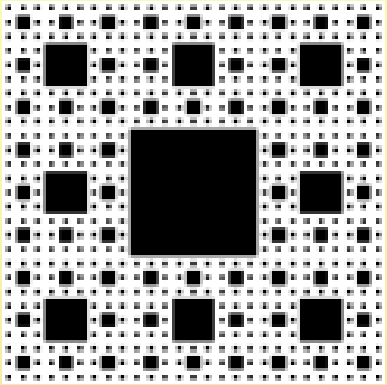
გ. არაერთმნიშვნელოვნება. Manifold - პრობლემები

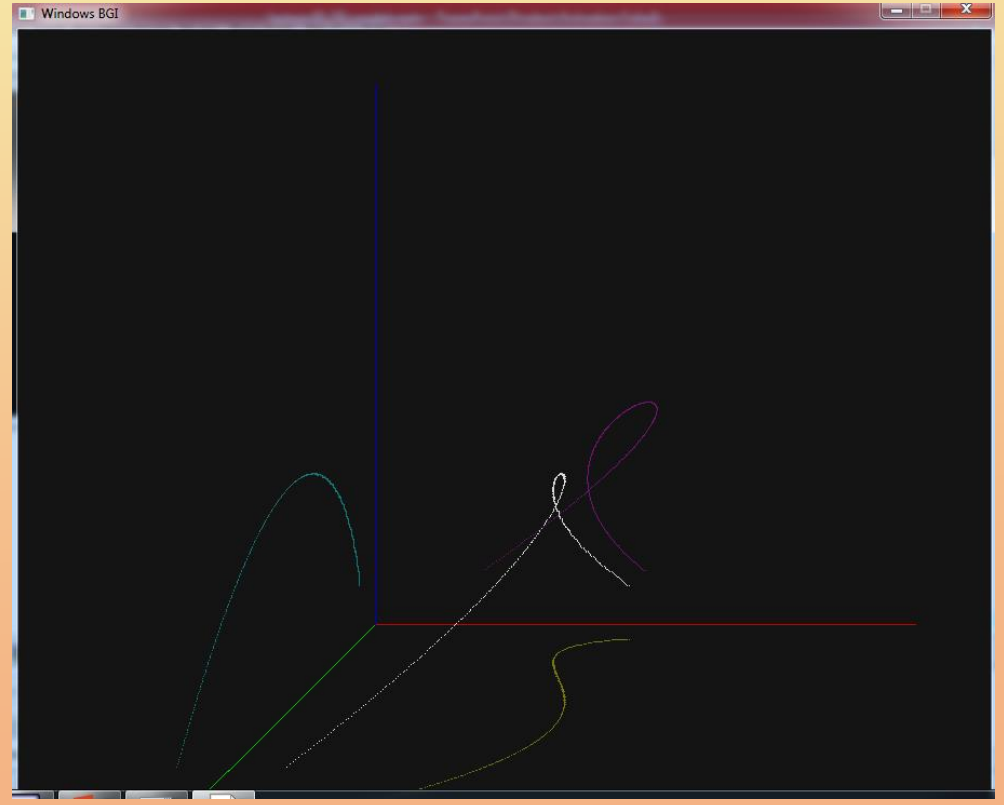






დავალება 1: დააპროგრამეთ „სერპინსკის ხალიჩის“ საილუსტრაციო პროგრამა.
ფრაქტალის ხაზვა ხდება სპეციალურ რეკურსიულ ფუნქციაში. მთავარი ფუნქცია გადასცემს ამ ფუნქციას რეკურსიის სიღრმეს.
https://en.wikipedia.org/wiki/Sierpinski_carpet





დავალება 2: 3-განზომილებიან სივრცეში მოცემულია 4 წერტილი. ამ წერტილების მიხედვით დახაზეთ ბეზიეს წირი და მისი სამივე პროექცია
3D სივრცისთვის გამოიყენეთ დიმეტრიული პროექცია

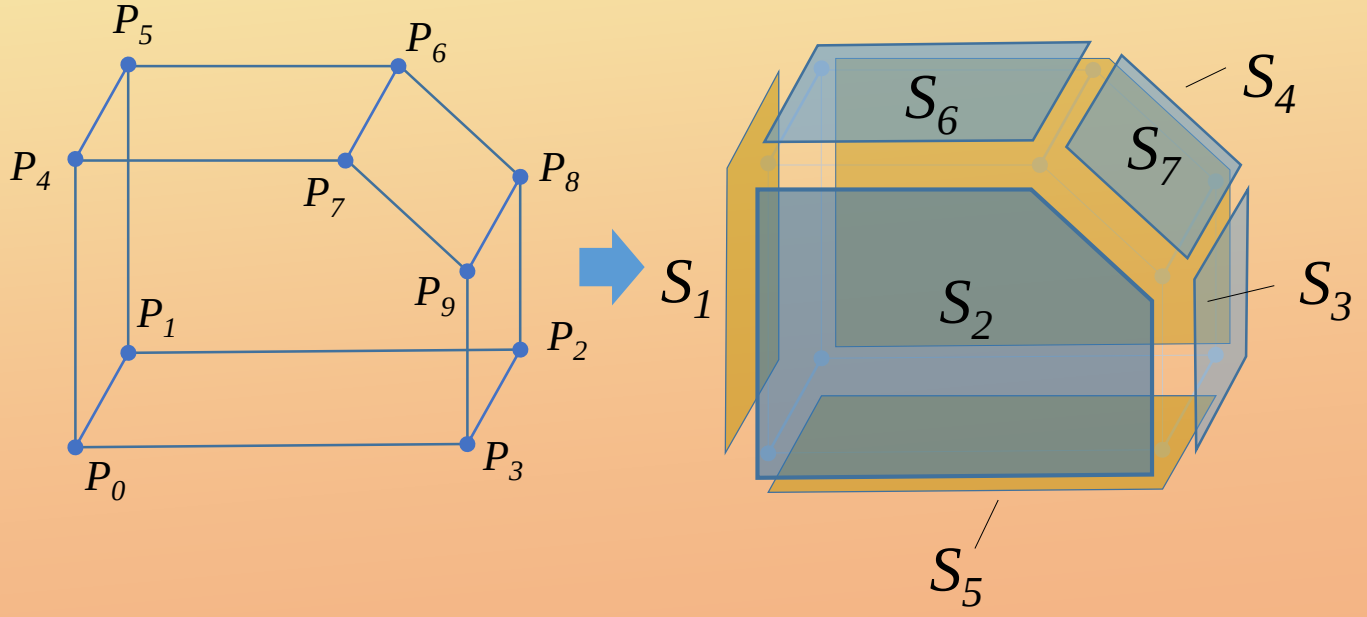
2. ზედაპირების მოდელები (surface models)

წერტილებისგან და ხაზებისგან შედგენილი მავთულებიანი მოდელებისგან განსხვავეებით, ზედაპირების მოდელები ინახავენ ინფორმაციას ასევე ზედაპირებზე

(AutoCAD – 3Dface,triangle meshes...)

მოდელი წარმოადგენს საყრდენ წერტილებს, რომლებზეც „გაჭიმულია“ ზედაპირის ელემენტები (სამკუთხედები, სპლაინ-ზედაპირები, ცარიელი ცილინდრები და სხვა).

ამ მოდელის შიგნით „მასალა“ არ არის, მოდელი წარმოადგენს ცარიელ გარსს.

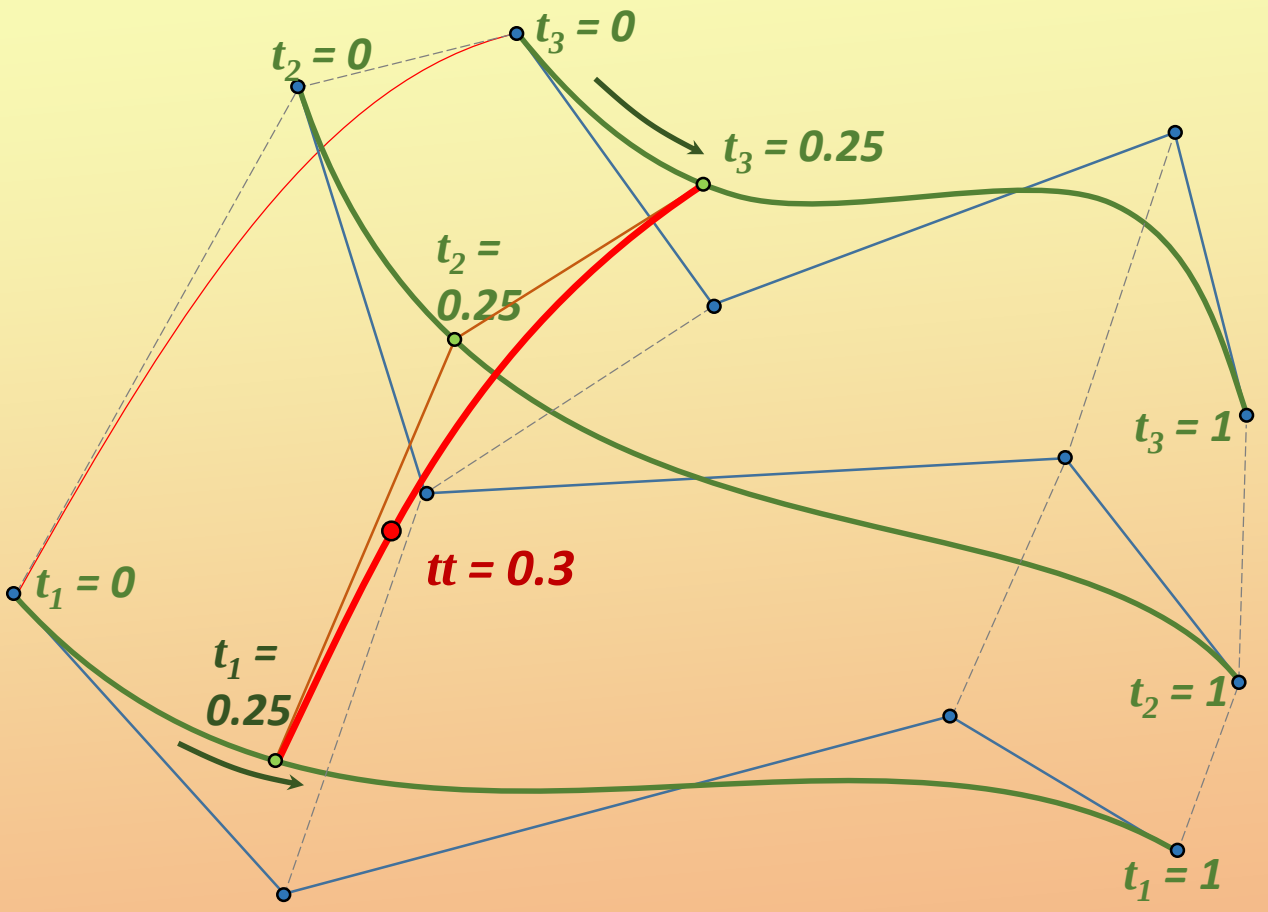


ყველაზე ხშირად ასეთი მოდელები სამკუთხედებისგან შედგება, მაგრამ ასევე შეიძლება სპლაინ-ზედაპირების გამოყენებაც:

ბეზიეს ზედაპირის ფორმა არის განპირობებული მისი საკონტროლო ბადის მიერ. ბადე თამაშობს ბეზიეს ზედაპირისთვის იმავე როლს, რაც საკონტროლო პოლიგონი ბეზის წირისთვის

ბეზიეს ზედაპირი (bezier patch) ეხება თავის საკონტროლო ბადეს ოთხ წერტილში - ბადის კუთხეებში

ბეზიეს ზედაპირის ვიზუალიზაცია ან თუნდაც ტრიანგულირება მარტივად ხდება: ბადის ერთიმიმასრთულებით ხდება ცალკეული ბეზიეს წირების გამოთვლა ფიქსირებყლი ბიჯით და პარამეტრით ***t***.



პარამეტრ ***t***-ს გარკვეული მნიშვნელობებისთვის (სურთზე ***t =0,25***) ვაერთიანებთ მრუდეების წერტილებს (სურათზე - მწვანე წერტილები) და მიღებულ პოლიგონზე ვანგარიშობთ ბეზიეს წირებს ***tt*** კოეფიციენტის მეშვეობით. t-ს და tt-ს მნიშვნელობების ყველა წყვილისთვის გაანგარიშებული x,y, და z - კოორდინატა ინახება [t][tt] მასივში

ზედაპირების მოდელები ადვილად ინახება მეხსიერებაში, მათი დამუშავება მარტივია. შესაძლებელია ასევე მათი გამოყენება ვირტუალურ რეალობაში (ზედაპირებისთვის ტექსტურების მინიჭება, მათი განათების გაანგარიშება და სხვა)

მინუსები: ზედაპირების მოდელები „ვერ ხედავენ“ რეალურ ობიექტებს. მოდელმა „არ იცის“ , რა ზედაპირებით არის შემოფარგლული 3D ობიექტი. შეუძლებელია ასეთი ობიექტების გაჭრა ან მათი მოცულობის გაანგარიშება. ეს არის მხოლოდ გარსი, შინაარსის გარეშე.

3. მოცულობითი მოდელები (volume models, Volumetric models, solid models)

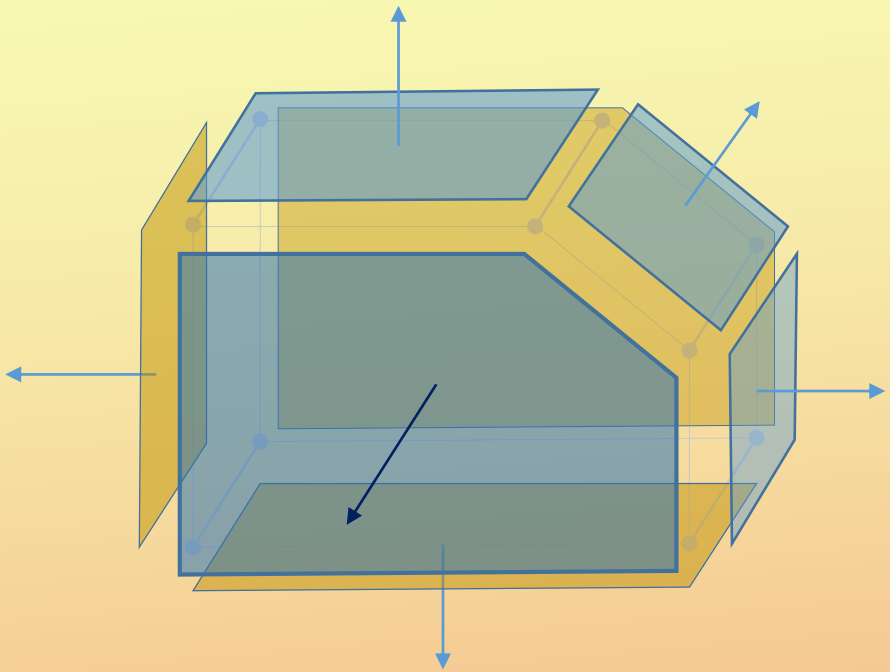
ზედაპირების მოდელისგან განსხვავებით, მოცულობითი მოდელები რეალურად ასახავს სამგანზომილებიან სხეულებს

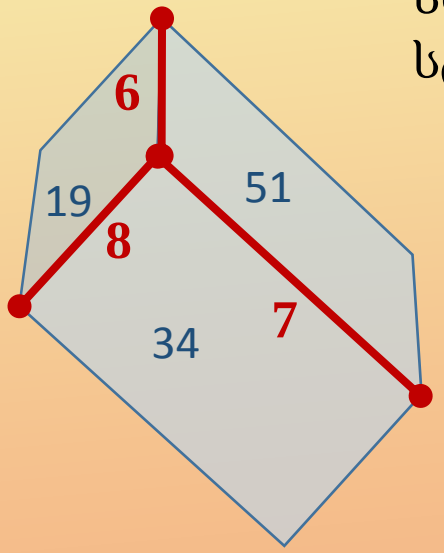
არსებობს მოცულობითი მოდელის აგების ორი პრინციპი:

1. Boundary Representation (B-Rep)
2. Constructive Solids Geometry (CSG)

Boundary Representation (B-Rep):

B-Rep მოდელი ნაწილობრივ ჰგავს ზედაპირების მოდელს, ოღონდ მის ტოპოლოგიაში დამატებულია მოცულობის, volume object-ს პარამეტრები და ასევე იმის ჩამონათვალი, თუ რომელ წიბოს რომელი ზედაპირი ემიჯნება ზოგჯერ ამ სქემას ეძახიან „ფრთებიანი წიბო“ (winged edge):



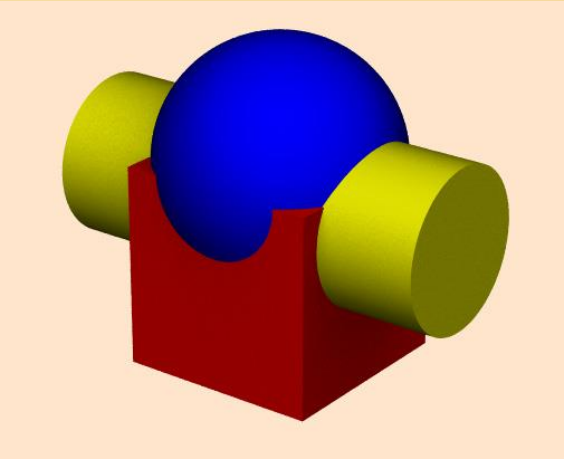


სხეულის საინფორმაციო სტრუქტურა (ტოპოლოგია)

6	19	51
7	34	51
8	19	34

+ დამატებითი ალგორითმები გარსის სისრულის კონტროლისთვის

Constructive Solids Geometry

სხეული იგება ცალკეული გრაფიკული პრიმიტივებზე (ელემენტებზე) ბულის ოპერაციების ჩატარების მეშვეობით

Boolean